NxN Rubik's Cube — Technical Basis

1. Overview

We build six solvers for two puzzles, the N×N×1 Rubik′s Square and the N×N×N Rubik′s Cube, for every N from 3 to 20, and benchmark them on one shared engine. Methods 1–4 are established techniques and meet the course requirement. Methods 5A and 5B are new learned solvers, aimed at a paper in the style of CayleyPy (NeurIPS 2025).

The gap we target. ML solvers today stop at 5×5×5 and train a separate model for each size. Classical solvers handle any N but produce long solutions. Nobody has a learned solver for N = 6 to 20, and God′s number is unknown for every N ⩾ 4.

#	Method	Approach	Sizes	Owner	What it gives the paper
1	IDA* + pattern databases	Exact heuristic search	3×3×3, small squares	Shree	Optimal ground truth
2	Reduction + Kociemba two-phase	Staged solving, one cluster at a time	All N	Sunaina	Standard practical baseline
3	Demaine parallel cluster algorithm	Group-theoretic, batched commutators	All N	Kashvi	Asymptotically optimal baseline, $\Theta(N^2/\log N)$
4	CayleyPy reproduction	Learned heuristic + beam search, one model per N	N = 3–5	Shaurya	Current ML state of the art
5A	Cluster-token transformer	Learned, one model for all N	All N	Sharanya	Headline: generalizes to sizes never trained on
5B	Learned search inside reduction stages	Learned per-stage heuristics + tiered beam search	All N	Gaurav	Guaranteed result: shorter solutions at large N
—	Shared engine, checker, benchmark harness	Infrastructure	All N	Gaurav	Makes every result comparable

Deliverables

1. Course: working code for all methods, a demo, and a report comparing them.
2. Paper: methods 5A and 5B with methods 1–4 as baselines, scaling plots across N, and God′s-number estimates.
3. Open source: the engine, all solvers, trained weights and the benchmark datasets.

One rule for everyone: every solver reads states from, and returns moves to, the shared engine in Section 3. The engine re-applies every returned solution and checks it is solved, so no method can report an unverified result.

2. Problem statement

Given any reachable scrambled state of an N×N×1 Rubik′s Square or an N×N×N Rubik′s Cube, with 3 ⩽ N ⩽ 20, return a move sequence that solves it. We want short solutions, found quickly, with one learned model per puzzle family rather than one per N.

Both problems are hard in a precise sense. Finding a shortest solution is NP-complete for the Square and the Cube (Demaine, Eisenstat and Rudoy, STACS 2018). The worst case, God′s number, grows as $\Theta(N^2/\log N)$ for both (Demaine et al., ESA 2011), but its constant is unknown.

2.1 Part A: the N×N×1 Rubik's Square

- **State:** N^2 cubies in an N×N grid, each with a top and a bottom colour. Border cubies also carry side stickers.
- **Moves:** 2N moves, a 180° flip of one row or one column. Every move is its own inverse, so a solution never repeats a move twice in a row.
- **Goal:** report both versions. Position-solved means every cubie is back in its home cell. Colour-solved means the top is all one colour and the bottom all the other. The NP-completeness result holds for both.
- **Size of the problem:** at least $6^{((\lfloor N/2 \rfloor - 1)^2)}$ reachable states, from the interior clusters alone (Demaine 2011, Theorem 2).

A cubie at (x, y) can only ever reach four cells: (x, y), (x, N−1−y), (N−1−x, y) and (N−1−x, N−1−y). The cubies sharing those cells form a **cluster**, and moves never mix clusters. Every cluster type must be handled:

Cluster type	Cubies	Exists when	How many	Notes
Corner	4	all N	1	Few reachable states; solve first
Edge	4	N ⩾ 4	$2(\lfloor N/2 \rfloor - 1)$	On the border rows and columns
Interior	4	N ⩾ 4	$(\lfloor N/2 \rfloor - 1)^2$	Exactly 6 reachable configurations each (Demaine, Lemma 1)

Cluster type	Cubies	Exists when	How many	Notes
Cross	2	odd N	about N	On the middle row and column; parity links them to the edge clusters (Lemma 3)
Center	1	odd N	1	Only its orientation changes

2.2 Part B: the N×N×N Rubik's Cube

- **State:** $6N^2$ stickers, one colour each.
- **Moves:** a quarter or half turn of any of the 3N slices. That gives 6N quarter-turn moves, or 9N moves when half turns count as one.
- **Move metrics:** report all three. QTM counts quarter turns, HTM counts any face or slice turn as one, and STM counts block turns. CayleyPy, DeepCubeA and Kaggle Santa each use a different one, so a result in only one metric cannot be compared with all baselines.
- **Goal:** each face one colour. For odd N the fixed centers define which colour goes where. For even N the solver may finish in any of the 24 orientations.
- **Size of the problem:** at least $(24!/(4!)^6)^{((\lfloor N/2 \rfloor - 1)^2)}$ reachable states, from the center orbits alone (Demaine 2011, Theorem 4).

Pieces move in **orbits**: a piece can only reach positions in its own orbit. Every orbit type must be handled:

Orbit type	Pieces	Exists when	Orbits for a given N	Constraint
Corners	8	all N	1	Twist sum ≡ 0 mod 3
Middle edges	12	odd N	1	Flip sum even; parity tied to corners
Wing edges	24	$N \geq 4$	$\lfloor (N-2)/2 \rfloor$	Orientation fixed by position; any permutation reachable
X-centers	24	$N \geq 4$	$\lfloor (N-2)/2 \rfloor$	4 stickers of each colour
+-centers	24	odd $N \geq 5$	$(N-3)/2$	4 stickers of each colour

Orbit type	Pieces	Exists when	Orbits for a given N	Constraint
Oblique centers	24	N ⩾ 6	the rest, in mirror-image pairs	4 stickers of each colour
Fixed centers	6	odd N	1	Define the colour scheme

In total there are $((N-2)^2 - [N \text{ odd}])/4$ center orbits. The orbit counts per N, which each solver must loop over:

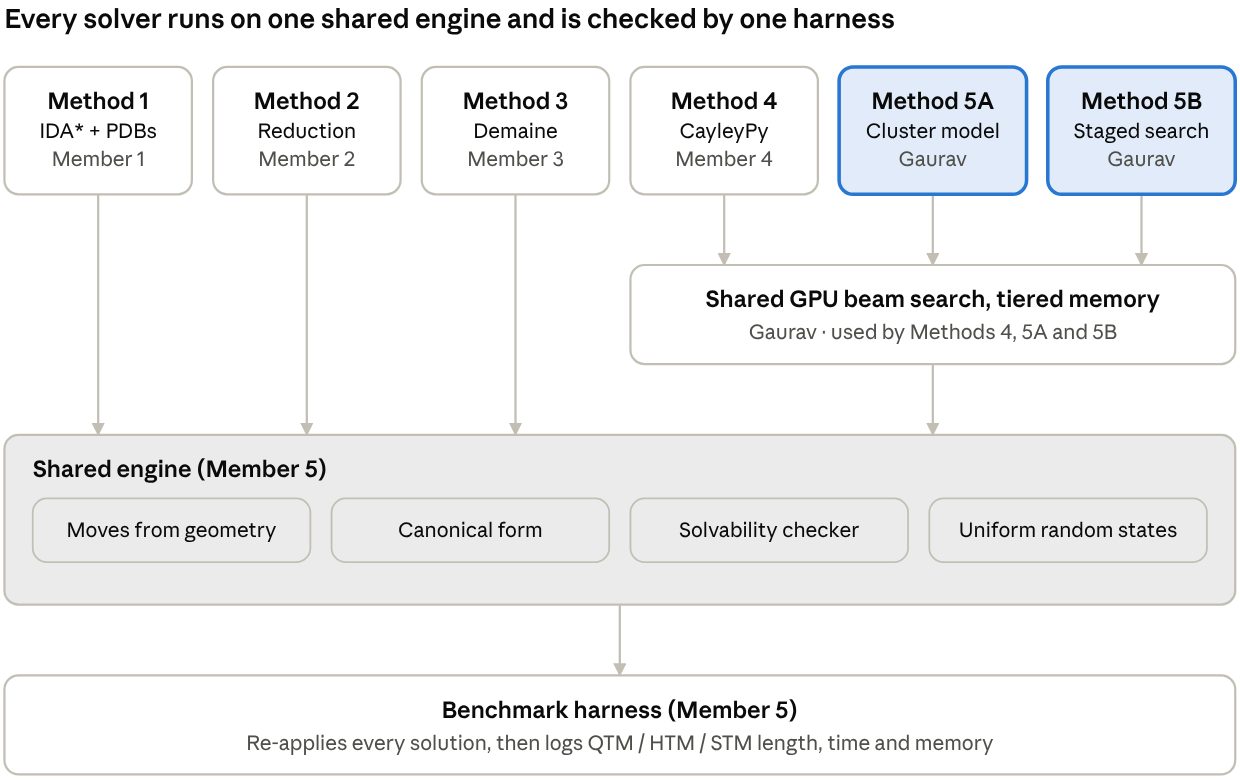
N	Stickers	Middle edges	Wing orbits	Center orbits
3	54	yes	0	0
4	96	no	1	1
5	150	yes	1	2
6	216	no	2	4
7	294	yes	2	6
10	600	no	4	16
20	2400	no	9	81

2.3 Cases every solver must cover

1. **Odd vs. even N.** Even N has no fixed centers and no middle edges, so the solver picks an orientation and must handle the two even-cube parities: OLL parity (one paired edge flipped) and PLL parity (two paired edges or two corners swapped).
2. **Identical center stickers.** Each center orbit has four interchangeable stickers per colour. This shrinks the state space and removes the permutation-parity constraint on centers.
3. **N = 3 and the smallest squares.** These are small enough for exact optimal solving, so they are where we measure each method's optimality gap.
4. **Large N.** At N = 20 a solution runs to thousands of moves and there are about 120 slice moves to choose from at each step. Any method whose cost grows with both is infeasible there.

3. Shared engine and benchmark (Gaurav)

One engine represents every puzzle as a permutation of sticker positions, so all six methods share the same moves, checks and metrics. Member 5 owns it; everyone else codes only against its interface.



project architecture · six solvers, one engine, one harness

Methods 1–3 call the engine directly; the three learned methods share one GPU beam search on top of it. Everything ends at the harness, which re-checks every solution.

3.1 State and moves

- **Puzzle spec:** a cuboid with dimensions (N, N, N) or (N, N, 1). Both puzzles come from the same code.
- **State:** a `uint8` array with one entry per sticker, holding its colour 0–5. Batches are a `(B, S)` tensor, where S is $6N^2$ for the cube.
- **Building moves from geometry, not by hand:** give every sticker a 3D position and a normal vector. A move rotates one slab of slices by 90° or 180° about x, y or z. Rotating the positions of the stickers in that slab gives a permutation array. Generate all moves this way for each N and cache them.

- **Applying a move:** `state = state[perm]` in NumPy, or `torch.gather` for a batch on GPU.
- **Move IDs:** a tuple (axis, layer, turn), with turn in {+1, −1, 2}. A converter prints standard notation such as `3Rw'` for export and for checking against other solvers.

3.2 Canonical move sequences

Moves on the same axis commute. So within any run of same-axis moves, sort them by layer and merge repeats of the same layer. This canonical form is used three ways:

1. Counting solution length fairly in every metric.
2. Pruning search: never expand a move that breaks canonical order.
3. Generating training scrambles without wasted steps (an ablation for methods 4, 5A and 5B).

3.3 Solvability checker

`is_reachable(state)` checks each orbit separately:

- **Cube:** corner twist sum ≡ 0 mod 3; for odd N, middle-edge flip sum even and corner parity equal to middle-edge parity; wing stickers consistent with their fixed orientation; exactly four stickers of each colour in every center orbit.
- **Square:** each cluster's configuration must be in its reachable set, found by breadth-first search over that cluster in isolation. Then apply the global edge-and-cross parity coupling from Demaine (2011), Lemma 3.

The wing and center rules in Section 2.2 are the expected constraints, not proven in this doc. Verify them before relying on them:

- [ ] 100,000 random scrambles for each N: the checker must accept every one.
- [ ] Single swaps or twists applied to solved states: the checker must reject every one that the theory says is unreachable.

3.4 Uniform random states

Random scrambles of fixed length are not uniform random states. Generate true random states orbit by orbit: shuffle each orbit, then fix twist, flip and parity so the checker passes. Benchmarks use both kinds, described in Section 10.

3.5 The interface every method implements

```
class Solver:
    name: str
    def __init__(self, puzzle: PuzzleSpec, budget: Budget): ...
    def solve(self, state: np.ndarray) -> SolveResult: ...

@dataclass
class SolveResult:
    moves: list[Move]      # in engine move IDs
    solved: bool           # set by the harness after re-applying moves, not
by the solver
    wall_time_s: float
    nodes_expanded: int
    peak_gpu_mb: float
    peak_host_mb: float
```

`Budget` caps wall time, node count and memory, so runs that do not finish are reported as failures rather than left running.

3.6 Benchmark harness and data format

- One JSON-lines file per (puzzle, N, dataset). Each row: `puzzle`, `N`, `state`, `scramble` (if generated by scrambling), `seed`, and `optimal_len` where Method 1 found it.
- `bench run --solver <name> --puzzle cube --N 3-20` writes one results row per instance: lengths in QTM, HTM and STM, time, nodes and memory.
- Every result records hardware, git commit and seed.

3.7 Stack and repo layout

- Python 3.11, NumPy and PyTorch (CUDA) for the engine and learned methods; Numba or C++ for the IDA* inner loop.
- Tests run in CI on every push: move-group identities (each move four times equals identity, a move then its inverse equals identity), checker property tests, and a smoke solve for each method.

```
engine/        puzzle spec, moves, canonical form, checker, random states
bench/         datasets, harness, metrics, plots
solvers/
  m1_idastar/  m2_reduction/  m3_demaine/  m4_cayleypy/  m5a_tokens/
m5b_stages/
search/        shared GPU beam search, tiered memory (Section 9)
experiments/   one config file per experiment in Section 10
```

4. Method 1: IDA* with pattern databases (Shree)

Method 1 finds provably shortest solutions where that is feasible, and so supplies the ground truth every other method is measured against. It also shows, with a measured curve, where exact search stops working.

4.1 Scope

- **3×3×3:** optimal solutions in HTM and QTM for the DeepCubeA test set, or a subset if time is short.
- **Rubik's Square:** full breadth-first search of the whole state space for the smallest N, which gives the exact God's number. Above that, IDA* for as large an N as the budget allows.
- **The finding to report:** the largest N, for each puzzle, at which IDA* still solves random states within the budget.

4.2 Algorithm

IDA* runs depth-first searches with a cost bound f = g + h, raising the bound each round until a solution is found. With an admissible h (one that never overestimates), the first solution found is optimal.

For the 3×3×3, use Korf's (1997) pattern databases: precomputed tables of exact distances for a subset of pieces.

Database	Pieces	Entries	Storage at 4 bits/entry
Corners	8 corners	$8! \times 3^7 = 88{,}179{,}840$	about 44 MB
Edges A	6 of the 12 edges	$12!/6! \times 2^6 = 42{,}577{,}920$	about 21 MB
Edges B	the other 6 edges	42,577,920	about 21 MB

The heuristic is h = max of the three lookups, which stays admissible. Build each table by breadth-first search backwards from the solved state, over that subset of pieces only.

For the Rubik's Square, use a per-cluster heuristic: h = the largest, over all clusters, of the moves that cluster needs on its own. This is admissible, because no solution can solve a cluster in fewer moves than the cluster needs in isolation.

4.3 Pruning

- Never apply the same face twice in a row.
- For opposite faces, which commute, allow only one order (the canonical form in Section 3.2).
- Square: never repeat a move, since every move is its own inverse.

4.4 Engineering

- Write the inner loop in C++ or Numba, not plain Python. Plain Python will be about 100× too slow for 3×3×3.
- Use a compact piece-level state (corner permutation and twist, edge permutation and flip) inside the search, not the sticker array.
- Pack pattern databases at 4 bits per entry and index them with permutation ranking.
- Cross-check a sample of optimal solution lengths against Rokicki's optimal solver (cube20src).

4.5 Done when

- [ ] Pattern databases built, with entry counts matching the table above.
- [ ] Optimal solutions for at least 100 3×3×3 instances, with mean length reported in HTM and QTM.
- [ ] Exact God's number for the smallest Rubik's Squares, from full breadth-first search.
- [ ] A plot of IDA* success rate against N for the Square, showing where it fails.

5. Method 2: Reduction + Kociemba two-phase (Sunaina)

Method 2 is the standard way to solve any N: reduce the big puzzle to a 3×3×3 stage by stage, then hand the result to a fast 3×3×3 solver. It works for every N and uses $O(N^2)$ moves.

5.1 Cube stages

1. **Centers.** Solve every center orbit so each face's centers are one colour. For odd N, build around the fixed centers. For even N, choose the colour of each face first, then keep to it. Use commutators: a sequence [A, B] = A B A′ B′ cycles three center pieces within one orbit and leaves everything else alone. Build a small library of these, parameterized by layer indices, so it works for any N.
2. **Edge pairing.** Group the wings of each orbit, plus the middle edge for odd N, into 12 composite edges. Pair them using slice moves that are undone afterwards so the centers stay solved, as in the standard slice-flip-slice method.
3. **Parity fixes.** Correct any wing orbit left with odd parity using a known sequence on that orbit's inner slices. For even N, these are the OLL and PLL parity cases from Section 2.3.
4. **3×3×3 stage.** The cube now behaves as a 3×3×3. Solve it with Kociemba's two-phase algorithm: min2phase (cs0x7f) or RubiksCube-TwophaseSolver (hkociemba).

5.2 Square version

Solve clusters one at a time using the short sequences from Demaine (2011), Lemma 2: at most six moves per interior cluster, leaving every other cluster unchanged. Solve the corner, edge and cross clusters with the O(N) procedure in Lemma 3. This is the one-cluster-at-a-time $O(N^2)$ algorithm that Method 3 parallelizes, so the Method 2 vs. Method 3 comparison isolates the effect of parallelism.

5.3 Engineering

- Keep each stage a separate function that takes a state and returns moves, so Method 5B can replace one stage at a time.
- Log moves per stage, so we can see where the length goes.
- Run the dwalton76 NxNxN solver on the same instances as an external baseline, and as a sanity check on our move counts.

5.4 Done when

- [ ] Solves 100% of benchmark instances for N = 3 to 20 on both puzzles.
- [ ] Every parity case reached at least once in testing, for both odd and even N.
- [ ] Move counts per stage reported for each N.

6. Method 3: Demaine's parallel cluster algorithm (Kashvi)

Method 3 implements the asymptotically optimal $O(N^2/\log N)$ algorithm from Demaine et al. (2011). The key idea: one slice move touches many clusters at once, so many clusters needing the same fix can share one move sequence. Build the Square version first, since it has only 6 interior configurations, then the Cube.

6.1 The batching trick (Square, Lemma 4)

Suppose every cluster in a set of columns X and rows Y is in the same configuration. Flip all columns in X, then all rows in Y, then X again, then Y again. Each cluster in X × Y sees exactly its own four-move fix. Every other cluster sees each of its moves twice, which cancels. So $|X| \times |Y|$ clusters are solved in $O(|X| + |Y|)$ moves instead of $O(|X| \times |Y|)$.

6.2 Full Square algorithm (Lemmas 5–6, Theorem 1)

1. Solve the corner, edge and cross clusters in O(N) moves (Lemma 3).
2. For each of the 6 interior configurations c, do the following.
3. Split the columns into groups of size $\ell = \frac{1}{2} \log_2 |Y|$.
4. Within a group, sort the rows by which subset S of the group's columns are in configuration c, and apply step 6.1 once for each subset S.
5. Total cost: $O(N^2/\log N)$ moves.

6.3 Cube version (Lemmas 7–12, Theorem 3)

1. **Parity:** fix each odd-parity edge cluster with one slice move, O(N) moves in total (Lemma 7).
2. **Edges:** solve each edge cluster with an O(1) sequence, O(N) moves in total.
3. **Interior clusters:** split the indices $\{1, \ldots, \lfloor N/2 \rfloor - 1\}$ into about $\sqrt{(N/2)}$ groups. For each ordered pair of different groups, apply the bulk construction of Lemmas 10–12, with each slice move expanded into one move per index in the group. Solve the clusters whose two indices fall in the same group one at a time.

The building block is Schütze's commutator S from the paper: ten moves that cycle exactly three cubies of one cluster and leave everything else unchanged. Choosing which faces count as front, left and up gives 24 variants, listed as permutations in Table 1 of the paper. Together they generate every even permutation of a cluster's 24 positions (Lemma 9).

6.4 The practical problem, and our fix

The theorem batches clusters that are in exactly the same configuration. But a cube cluster has about $24!/(4!)^6 \approx 3 \times 10^{15}$ configurations, so at $N \leqslant 20$ two clusters almost never match, and batching by configuration does almost nothing.

Instead, batch at the level of 3-cycles, as the KMCoders Kaggle Santa 2023 solution did:

1. Write each cluster's solution as a sequence of basic 3-cycles from the 24 variants.
2. In each round, choose one 3-cycle type and apply it at once to every cluster whose next needed step is that type, using the bulk moves of Lemma 10.
3. Repeat until every cluster is solved.

Which 3-cycle to apply in each round is a scheduling problem. Start with a greedy rule (pick the type the most clusters need next). Learning a better schedule is a stretch goal and a possible extension of the paper.

6.5 Done when

- [ ] Square version solves all instances for N = 3 to 20.
- [ ] Cube version solves all instances for N = 3 to 20.
- [ ] Plot of moves divided by $N^2/\log N$ against N, for both puzzles.
- [ ] Direct comparison with Method 2 on the same instances, to show what parallelism saves.

7. Method 4: CayleyPy reproduction (Shaurya)

Method 4 reproduces the current ML state of the art (Chervov et al., NeurIPS 2025) inside our engine. It is the per-N baseline that 5A must match while using one model for all N.

7.1 How it works

1. **Training data from random walks.** Start at the solved state and apply k random moves, never undoing the previous move (a non-backtracking walk). Record the pair (state, k). Generate fresh walks every epoch; it costs almost nothing.
2. **Network.** A ResMLP: one-hot input of size $6 \times 6N^2$, two linear layers with BatchNorm and ReLU, then residual blocks, then one output. The output predicts k, the expected number of random steps from solved, which acts as a distance estimate.
3. **Training.** MSE loss, Adam at learning rate 0.001, 1M fresh examples per epoch, FP32 training and FP16 inference.
4. **Beam search.** From the scramble, expand every move, score all children with the network, keep the W lowest, drop duplicates by hash, and repeat until the solved state appears or 200 steps pass.
5. **Ensembles (optional).** Several independently trained networks each search, and the shortest solution wins.

7.2 Settings from the paper

Puzzle	Max walk length	Model	Paper result
3×3×3	26	10 layers, 4M parameters	21.14 QTM average at $W = 2^{18}$ on DeepCubeA
4×4×4	45	10 layers, 4M parameters	48.98 average at $W = 2^{24}$ on Santa 2023
5×5×5	65	10 layers, 4M parameters	92.16 average, ensemble of 69 agents

The paper's finding to reproduce: solution length falls roughly linearly in log W, and adding training data beyond about 8 billion states barely helps.

7.3 Our runs

- Use the authors' code (github.com/k1242/cayleypy-cube) as the reference, but wrap it in our `Solver` interface so it reads benchmark files and is re-checked by the harness.
- Cube: N = 3, 4, 5. Expect a smaller W than the paper's A100 runs; report the length-versus-W curve for the widths we can afford.

- Square: train one model per N for N = 3 to 8. This is new data the paper does not have, and gives 5A a per-N baseline on the Square.

7.4 Done when

- [ ] The 3×3×3 average at W = 2^{18} is within about 0.5 moves of the paper's 21.14 QTM.
- [ ] A length-versus-W curve for 3×3×3 and 4×4×4.
- [ ] Per-N Square models for N = 3 to 8, evaluated on our benchmark.
- [ ] Training time and peak GPU memory recorded for every run.

8. Method 5A: one model for every N (Sharanya)

Method 5A replaces the per-N sticker network with a network that reads the puzzle as a set of clusters. The same weights then apply to any N, so we can train on small cubes and test on sizes never seen in training. If it works, this is the paper's headline: the first ML solver beyond 5×5×5.

8.1 Why clusters

A $6N^2$-sticker one-hot input has a different size for every N, so CayleyPy needs a new model per size. But the puzzle is made of a few repeated piece types (Section 2): every x-center orbit behaves alike whatever N is, and so does every wing orbit. A network built from those repeated parts can, in principle, reuse what it learned at N = 6 when it sees N = 15.

8.2 Input: one token per cluster

- **Cluster content:** the colours at the cluster's positions, one-hot: 24 positions × 6 colours for a cube orbit, 4 × 2 for a Square cluster.
- **Cluster type:** a learned embedding for corner, middle edge, wing, x-center, +-center or oblique.
- **Position in the cube:** the orbit's layer indices divided by N, so a layer halfway in looks the same at every size. Also add the move-slices that touch it, as indices divided by N.
- **Size:** N itself as an extra input, so the model can scale its estimates.

8.3 Network

- A transformer encoder over the cluster tokens (about 4–8 layers, width 256), with no positional order, since the clusters form a set.

- **Additive head:** each token predicts its own cost, the costs are summed, and a small global correction is added. A sum matches the structure of the problem, since distance grows with the number of clusters, and it extends naturally to larger N.
- *Move head (Q style):** add one token per slice move. Each move token attends to the cluster tokens it would change and outputs the predicted distance after that move. Scoring all about 9N moves then takes one forward pass, instead of building and scoring every child state. This follows Agostinelli et al.'s Q* search.

8.4 Training

- Random walks from solved, as in Method 4, on N in {3, 4, 5, 6, 7}. Longer walks for larger N.
- Target: steps from solved. Predict it divided by $N^2/\log N$ so the scale stays similar across N, then multiply back.
- Augment with the 24 rotations of the cube and permutations of colour labels; the true distance does not change under either.

8.5 Search

Beam search as in Method 4, using the move head to score children. For large N, a single search over thousands of steps may be too expensive. If so, run 5A inside the stages of Method 5B: 5A becomes the per-stage heuristic, and 5A and 5B merge into one system.

8.6 Go / no-go test: two weeks, on the Square

1. Train on Rubik's Squares with N = 4 to 8.
2. Test on N = 10, 14 and 20.
3. **Pass:** solves 100% of N = 10 instances in fewer moves than Method 2. Then build the Cube version.
4. **Fail:** keep 5B as the main contribution and report 5A's result as a negative finding, which is still worth stating in the paper.

8.7 Ablations

- Cluster tokens vs. sticker one-hot input, at the same parameter count.
- With vs. without position features.
- Training sizes: {3–5}, {3–7}, {3–9}.
- Additive head vs. plain pooled head.

- Move head vs. scoring every child.

9. Method 5B: learned search inside reduction stages (Gaurav)

Method 5B keeps Method 2's stage structure but replaces each stage's hand-written sequences with a learned heuristic plus beam search. Each stage only has to solve a few orbits, so its search problem is small enough to scale to N = 20. This is the result the paper can count on even if 5A fails.

9.1 Stages

1. **Centers,** a few orbits at a time, ordered from the outermost layer inward.
2. **Wing pairing,** one wing orbit at a time, including its parity.
3. **3×3×3 stage,** solved by min2phase as in Method 2. It is already near-optimal, so learning it gains little.

Each stage has an exact goal test: the orbits it targets are solved, and every orbit solved in an earlier stage is still solved.

9.2 Per-orbit heuristics

Looking only at one orbit's stickers gives a much smaller puzzle: the moves still act on it, but everything else is ignored. Its distance to solved is a lower bound on the moves the real stage needs for that orbit.

- Train one heuristic per orbit type (x-center, +-center, oblique, wing) on random walks of that projected puzzle, as in Method 4. Only the orbit's 24 stickers are input, so the networks are small and fast.
- The orbit's layer index changes which slice moves act on it, but not how it behaves, so one network per orbit type covers every N.
- **Stage heuristic:** the sum of the per-orbit heuristics over this stage's targets and every earlier stage's orbits. The second part penalizes breaking work already done. A sum is not admissible, but beam search does not need admissibility.

The per-orbit networks are exactly 5A's tokens seen one at a time. If 5A passes its go / no-go test, its model can replace the per-orbit sum here directly.

9.3 Tiered beam search (the systems contribution)

CayleyPy found solution length falls roughly linearly with log W. Wider beams are therefore the most direct lever on solution quality, and GPU memory is what limits them.

- **Bit-packed states:** 3 bits per sticker. At N = 20 that is 2400 × 3 bits = 900 bytes per state, so a beam of 2^{24} states is about 15 GB.
- **Store less per state:** keep a 64-bit hash, the score and a back-pointer (parent index plus move ID) for every beam entry. Rebuild full states only for the chunk being expanded.
- **Tiers:** the current chunk on GPU, the rest of the beam in pinned host memory, and spill to NVMe beyond that. Overlap transfers with compute using CUDA streams.
- **Duplicates:** a GPU hash table of state hashes. Partition the beam by hash range so each chunk can be deduplicated independently.
- **Top-W selection:** streaming partial sort across chunks, merging into one global top W.

The same engine also serves Method 4 and 5A, so every learned method benefits.

9.4 Done when

- [ ] Solves 100% of instances for N = 3 to 20.
- [ ] Shorter mean solutions than Method 2 and Method 3 at every N ⩾ 6.
- [ ] Tiered search runs a beam at least 16× wider than GPU-only search on the same GPU.
- [ ] Throughput (states per second), peak memory per tier and solution length reported against W.

10. Evaluation

Every method runs on the same instances through the same harness, and every claim in the paper maps to one experiment below.

10.1 Datasets

- **DeepCubeA 3×3×3 test set:** the standard set for comparing ML solvers.
- **Kaggle Santa 2023 cube puzzles:** best-known competition solutions to beat. Check which cube sizes it actually includes before relying on it for large N.
- **Our scrambles:** for each N and puzzle, instances scrambled to fixed depths (short, medium, long), to measure behaviour as difficulty grows.

- **Our random states:** uniform random reachable states from Section 3.4, the true average case.
- **Optimal set:** instances Method 1 solved optimally, for measuring optimality gaps.
- **Size per N:** 100 instances for N ⩽ 7 and 20 for N > 7, fixed by seed and shared by every method.

10.2 Metrics

- Success rate within the budget.
- Solution length in QTM, HTM and STM: mean with 95% confidence interval, and maximum.
- Optimality gap: length minus optimal, where the optimum is known.
- Length divided by $N^2/\log N$, to compare against the theoretical growth rate.
- Wall time, nodes expanded, peak GPU and host memory, and training cost in GPU-hours.

10.3 Experiments

ID	Question	Methods	Sizes
E1	How close to optimal is each method?	1 (reference), 2, 3, 4, 5A, 5B	3×3×3, small Squares
E2	Which method gives the shortest solutions at each size?	2, 3, 4, 5A, 5B	N = 3 to 20, both puzzles
E3	Does length grow as $N^2/\log N$, and with what constant?	2, 3, 5B	N = 3 to 20
E4	How does length change with beam width W?	4, 5A, 5B	3, 5, 10, 20
E5	Does 5A generalize to sizes it never saw?	4 (per-N), 5A	train ⩽ 7, test 8 to 20
E6	How much does tiered memory widen the beam, and what does it cost in speed?	search engine	10, 20
E7	What are the best upper bounds we can give for God's number?	best solver per N, plus counting lower bounds	N = 4 to 20

ID	Question	Methods	Sizes
E8	Does removing commuting-move redundancy from training scrambles help?	4, 5A, 5B	3 to 10

10.4 God's-number estimates (E7)

- **Upper bound evidence:** the longest solution our best solver needs over many uniform random states, plus states found by searching for the hardest positions. These are estimates, not proofs, and the paper must say so.
- **Lower bound:** the counting argument from Demaine (2011), Theorems 2 and 4, computed exactly for each N.
- **Exact values:** for the smallest Squares, from Method 1′s full breadth-first search.

10.5 Reporting rules

- Every number in the paper comes from a results file produced by the harness at a recorded git commit.
- Report hardware alongside every timing.
- A method that fails an instance counts as failed. Never drop failures to improve an average.

11. Timeline, ownership and team rules

The plan runs ten weeks from kickoff. The weeks shift once we know the project deadline (an open question below).

11.1 Phases

1. **Weeks 1–2: engine first.** Member 5 delivers the engine, checker and harness for both puzzles, with the interface frozen by the end of week 1. Everyone else reads Sections 2–3, studies their method′s paper and writes a 3×3×3 or small-Square prototype against the draft interface. Gaurav runs the 5A go / no-go test on the Square (Section 8.6).
2. **Weeks 3–5: Square for everything.** Each method solves the Rubik′s Square for N = 3 to 20. The Square is quicker to debug, and every method′s result there is already a usable comparison.

3. **Weeks 5–8: Cube for everything.** Each method extends to the N×N×N cube. Gaurav builds 5B and the tiered beam search, then 5A on the cube if the go / no-go test passed.
4. **Week 8: freeze.** No new features. All benchmark runs start from tagged commits.
5. **Weeks 9–10: experiments and writing.** Run E1–E8, produce the plots, write the course report and the paper draft.

11.2 Team rules

- One repo. One branch per method. Merge to main only with a passing CI run and one reviewer.
- Nobody edits `engine/` except through a pull request Member 5 approves, since every result depends on it.
- Commit results files only from the harness, never hand-edited.
- Weekly 30-minute sync: each owner shows one number from the harness.

11.3 References

- Demaine et al., Algorithms for Solving Rubik's Cubes (ESA 2011): cluster structure, $\Theta(N^2/\log N)$ bounds, Schütze commutator.
- Demaine, Eisenstat and Rudoy, Solving the Rubik's Cube Optimally is NP-complete (STACS 2018)
- Chervov et al., A Machine Learning Approach That Beats Rubik's Cubes (NeurIPS 2025) and arXiv version; code at k1242/cayleypy-cube
- CayleyPy library
- DeepCubeA code and DeepXube
- Agostinelli et al., Q* search
- EfficientCube
- dwalton76 NxNxN solver
- min2phase (cs0x7f) and Kociemba's RubiksCube-TwophaseSolver
- KMCoders Kaggle Santa 2023 solution
- magiccube NxNxN simulator
- Korf, Finding Optimal Solutions to Rubik's Cube Using Pattern Databases (AAAI 1997); Rokicki et al., God's number is 20